

# SERVEUR | CYBERSÉCURITÉ | SSH | DEVOPS | LINUX | UBUNTU

## SSH (Secure Shell): Un guide pour débutants

Un guide complet sur les bases de SSH, les connexions sécurisées, la gestion des clés et les techniques avancées pour le développement moderne.

[bektiaw](#)



## SSH Essentials: Connect Like a Pro

**Connaître SSH – c’est un must ! Pour les serveurs, GitHub, et au-delà.** De nombreux développeurs pensent que SSH est uniquement pour les administrateurs du système, mais en réalité, c’est un outil de base pour quiconque écrit et expédie du code. Que nous accédions à un serveur cloud, que nous installions des pipelines CI/CD ou que nous nous authentifiions avec GitHub, SSH joue un rôle essentiel dans les coulisses. C’est une compétence fondamentale que nous utiliserons quotidiennement.

Dans cet article, nous couvrirons l’essentiel de SSH, nous aidant à devenir confiants dans la connexion à distance, la poussée du code et la gestion de notre flux de travail de développement comme un pro.

### SSH (Secure Shell)

#### Qu'est-ce que SSH ?

SSH est un protocole de réseau cryptographique utilisé pour exploiter les services réseau de manière sécurisée sur un réseau non sécurisé.

#### Pourquoi SSH ?

SSH est la méthode standard pour se connecter en toute sécurité aux machines et services distants. En cryptant toutes les données, SSH protège nos informations contre les pirates et les écouteurs, assurant une communication sûre sur Internet.

## Cas d'utilisation courants de SSH

- Administration de serveur à distance
- Transferts de fichiers sécurisés (SCP, SFTP)
- Transfert et tunnels portuaires
- Opérations Git
- Déploiements automatisés

## Débuter avec SSH

### Connexion à un serveur distant (avec un mot de passe)

L'une des utilisations les plus courantes et les plus importantes de **SSH** est la connexion à un serveur distant. Le moyen le plus élémentaire de se connecter est avec un nom d'utilisateur et l'adresse du serveur (**un IP** **address** ou **un domain name**) avec commande **ssh** **<username>@<server\_address>**

```
ssh root@192.168.1.10
```

- **ssh**— Démarrer une connexion SSH.
- **root**— Le nom d'utilisateur que nous voulons connecter comme sur le serveur distant.
- **192.168.1.10**— C'est l'adresse IP du serveur distant auquel nous essayons de nous connecter.

Lors de la première connexion, nous verrons une invite de vérification des empreintes digitales comme celle-ci. Type **yes** pour continuer.

```
The authenticity of host 'example.com (93.184.216.34)' can't be
established.
ECDSA key fingerprint is
SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.
Are you sure you want to continue connecting (yes/no)?
```

Il nous sera alors demandé d'entrer dans le **password** pour le serveur distant. Si le mot de passe est correct, nous serons connectés et pouvons commencer à taper des commandes sur ce serveur, tout comme nous l'utilisons directement.

### Spécifier le port (par défaut est 22)

Nous pouvons également spécifier un port si le serveur n'utilise pas le port par défaut 22.

```
ssh -p 2222 root@192.168.1.10
```

- **-p 2222**— Se connecter en utilisant le **port 2222** au lieu du par défaut (22).

## Transfert de fichiers

**SSH** n'est pas seulement pour l'accès à distance, il peut également être utilisé pour transférer des fichiers en toute sécurité entre notre machine locale et un serveur distant.

### scp (Copie sécurisée)

**scp** Fonctionne comme la norme `cp` commandement.

Copiez un fichier de notre machine locale à un serveur distant

```
scp local/file.txt root @192.168.101.10 :/remote/
```

- `local/file.txt`– Chemin vers le fichier sur notre machine locale.
- `root@192.168.1.10`– Connexion au serveur à distance (user@IP).
- `/remote/`– Répertoire de destination sur le serveur distant.

Copiez un fichier à partir d'un serveur distant vers notre machine locale

```
scp root @192.168.110.10 :/remote/file.txt /local
```

- `/remote/file.txt`– Chemin vers le fichier sur le serveur distant.
- `/local`– Répertoire de destination sur notre machine locale.

### sftp (Secure File Transfer Protocol)

**sftp** fournit une session interactive similaire à un **client FTP**.

```
sftp root@192.168.1.10
```

- Une fois connecté, nous pouvons utiliser des commandes comme `ls`, `cd`, `get` (à télécharger), et `put` (à télécharger).

### Exécuter une commande unique

Nous pouvons exécuter une commande sur un serveur distant sans démarrer une session interactive complète.

```
ssh username@remote_host 'uptime -p'
```

Cela va se connecter, exécuter le `uptime -p` commande sur le serveur distant, imprimer la sortie et déconnecter.

## Authentification À Base De Clés: La Voie Sécurisée

Au lieu de taper notre mot de passe à chaque fois que nous nous connectons à un serveur, nous pouvons utiliser l'authentification basée sur les clés SSH. L'utilisation des clés SSH est beaucoup plus sécurisée que l'utilisation de mots de passe et est la méthode recommandée pour l'authentification.

Comment configurer l'authentification basée sur les clés:

## 1. Générer une paire de clés SSH sur notre machine locale

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

- **ssh-keygen**— Démarre le processus de génération de clés.
- **-t ed25519**— Spécifie le type de clé. Ed25519 est un algorithme moderne et sécurisé qui est préféré aux types plus anciens **comme RSA**.
- **-C "your\_email@example.com"**— Ajoute une étiquette (commentaire) à la clé, souvent notre email, pour l'identifier ultérieurement.

On nous demandera:

- Où enregistrer la clé (la valeur par défaut est très bien)
- Créer une phrase de passe. Une phrase de passe crypte notre clé privée sur notre disque local, ajoutant une couche de sécurité supplémentaire. C'est fortement recommandé.

Cette commande va générer deux fichiers dans notre `~/.ssh/` répertoire:

- `~/.ssh/id_ed25519`— **Notre clé privée** (gardez cela sécurisé, ne le partagez jamais !).
- `~/.ssh/id_ed25519.pub`— **Notre clé publique** (c'est ce que nous partageons avec les serveurs ou GitHub).

Nous pouvons également générer la clé SSH avec un nom personnalisé

```
ssh-keygen -t ed25519 -C "your_email@example.com" -f ~/.ssh/custom_key_name
```

- **-f ~/.ssh/custom\_key\_name**— Définissez le nom de fichier et le chemin d'accès pour la nouvelle clé.

## 2. Copier la clé publique dans le serveur

```
ssh-copy-id root@192.168.1.10
```

- **ssh-copy-id**— Un outil qui installe notre clé publique sur un serveur distant (annexer `authorized_keys` fichier sur le serveur distant)

Nous pouvons également utiliser `ssh-copy-id` avec la clé spécifique:

```
ssh-copy-id -i ~/.ssh/custom_key_name.pub root@192.168.1.10
```

- **-i ~/.ssh/custom\_key\_name.pub**— Raconter **ssh-copy-id** exactement quelle clé publique copier.

### 3. Se connecter sans mot de passe

Après avoir configuré l'authentification basée sur les clés et copié notre clé publique sur le serveur, nous pouvons nous connecter sans entrer votre mot de passe.

```
ssh root@192.168.1.10
```

Si tout est correctement configuré, nous serons connectés immédiatement en utilisant notre clé SSH, et aucune invite de mot de passe n'apparaîtra.

#### Note:

Si nous fixons une phrase de passe pour notre clé, nous serons invités à l'entrer. Sinon, nous serons connectés immédiatement sans mot de passe.

## Agent SSH

**SSH Agent** est un programme qui contient nos clés SSH privées en mémoire. Il nous aide à utiliser nos touches SSH sans saisir la phrase de passe à chaque fois que nous nous connectons à un serveur.

### 1. Démarrer l'agent

```
eval "$(ssh-agent -s)"
```

- `eval "$(ssh-agent -s)"`— Démarre l'agent SSH en arrière-plan. Configure les variables d'environnement comme `SSH_AGENT_PID` et `SSH_AUTH_SOCK`. Donc notre coquille sait comment parler à l'agent.

### 2. Ajouter la clé à l'agent

```
ssh-add ~/.ssh/id_ed25519
```

Nous pouvons également ajouter la clé avec la durée de vie

```
ssh-add -t 3600 ~/.ssh/id_ed25519
```

- `-t 3600`— Définit une expiration de 3600 secondes (1 heure) pour cette touche. Après 1 heure, la clé est automatiquement retirée de l'agent SSH.

### 3. Liste de clés ajoutées

```
ssh-add -l
```

Nous pouvons également supprimer toutes les clés par exemple si nous voulons effacer nos clés rapidement avant de quitter un ordinateur partagé ou si nous voulons réinitialiser les clés chargées.

```
ssh-add -D
```

# Configuration SSH

## Configuration du client

Nous pouvons personnaliser la façon dont nous nous connectons aux serveurs en configurant le `~/.ssh/config` fichier. Ce fichier rend la gestion de plusieurs connexions SSH plus facile et plus sécurisée.

`chat ~/.ssh/config`

```
Host myserver
  HostName example.com
  User username
  Port 22
  IdentityFile ~/.ssh/id_ed25519
```

Configuration de base de l'hôte, / Configuration simple pour un seul serveur.

`Serveur hôte1`

`Nom d'hôte 192.168.1.10`

`Source de l'utilisateur`

`Port 22`

`IdentityFile ~/.ssh/id_ed25519_server1`

- **Host**— L'alias ou le surnom que nous donnons à un serveur.
- **HostName**— L'adresse IP ou le domaine du serveur.
- **User**— Le nom d'utilisateur du login (Facultatif, par défaut: utilisateur local actuel).
- **Port**— Le port SSH (facultatif, par défaut:22).
- **IdentityFile**— spécifie quelle clé SSH utiliser (optionnel, par défaut: clés standard comme `~/.ssh/id_rsa`, `~/.ssh/id_ed25519`, etc)

Maintenant, au lieu de taper la commande complète, nous pouvons simplement **utiliser l'alias** pour nous connecter à notre serveur distant : `ssh myserver`

- **myserver**— l'alias que nous avons défini.

## Plusieurs serveurs avec des paramètres similaires

Appliquer les mêmes paramètres à plusieurs hôtes.

`Hébergeur webserver1 webserver2 webserver3`

`Déployer un utilisateur`

`Port 22`

`IdentityFile ~/.ssh/id_ed25519_deploy`

## Hôte de saut

Connectez-vous à un serveur interne via un hôte de saut/bastion utilisant `ProxyJump`.

```
Hôte saut-hôte
HostName jump.example.com
Alice utilisateur
IdentityFile ~/.ssh/id_ed25519_jump
```

## Wildcard

Nous pouvons utiliser `*` comme une wildcard pour appliquer des paramètres à plusieurs hôtes qui partagent un motif.

### Hôte \*

```
ServerAliveInterval 60
ServerAliveCountMax 3
La compression oui
```

### Hôte \*.example.com

```
Utilisateur admin
IdentitéFile ~/.ssh/id_ed25519_example
```

- **Host \*** — S'applique à toutes les **connexions SSH**.
- **Host \*.example.com** — correspond à tout hôte qui se termine par **.example.com**
- **ServerAliveInterval 60** — Toutes les 60 secondes, notre ordinateur envoie un petit message au serveur pour vérifier s'il est toujours connecté. Cela permet d'éviter que la connexion ne baisse en raison d'inactivité ou de problèmes de réseau.
- **ServerAliveCountMax 3** — Si le serveur ne répond pas 3 fois de suite, notre ordinateur cessera d'essayer de fermer la connexion.

## Considérations clés pour `~/.ssh/config` et SSH Keys

- **SSH Reads Config Top to Bottom** — SSH applique la première correspondance `Host` bloc il trouve lors de la connexion. Donc, placez nos blocs hôtes plus spécifiques avant ceux génériques comme `Host *`.
- **Spécifier la clé de droite pour chaque hôte** — Utilisez le **IdentityFile** option pour indiquer à SSH quelle clé privée utiliser pour chaque serveur ou groupe de serveurs.
- **Évitez d'utiliser trop de clés automatiquement** – Si nous avons de nombreuses clés chargées dans notre **agent SSH**, cela pourrait les essayer toutes et causer des retards ou des défaillances d'authentification.

Réglage **IdentitiesOnly yes** qui fait **que SSH** essaie uniquement les clés spécifiées dans **IdentityFile** ou ligne de commande.

- **Utilisation `AddKeysToAgent`(macOS et Modern SSH)** — Ajoutez automatiquement des touches à notre agent SSH lorsqu'il est utilisé.
- **Utilisation `IgnoreUnknown` pour Portability** — `IgnoreUnknown` évite les erreurs si certaines options ne sont pas prises en charge sur le **client SSH** de notre système. Cela rend notre fichier de configuration fonctionnant en douceur à travers différentes versions OS et SSH.

Exemple: `IgnoreUnknown UseKeychain, AddKeysToAgent`

## Configuration du serveur

La configuration du serveur SSH est située à `/etc/ssh/sshd_config`. Ce fichier contrôle le fonctionnement du serveur SSH, y compris les paramètres de sécurité, les méthodes d'authentification et les options de connexion.

### Paramètres communs et importants

- **Port**— Le port TCP SSH écoute (par défaut: `22`). Exemple: `Port 2222`.
- **PermitRootLogin**— Contrôle si l'utilisateur root peut se connecter directement. Valeurs: `yes`, `no`, `prohibit-password`, `without-password`. Exemple: `PermitRootLogin no` (recommandé pour la sécurité).
- **PasswordAuthentication**— Autoriser ou refuser la connexion au mot de passe. Exemple: `PasswordAuthentication no` (Forces d'utilisation des clés).
- **PubkeyAuthentication**— Activer l'authentification à clé publique. Exemple: `PubkeyAuthentication yes`
- **MaxAuthTries**— Tentatives d'authentification maximales autorisées avant de se déconnecter. Exemple: `MaxAuthTries 3`
- **AllowUsers/ AllowGroups**— Restreindre les utilisateurs ou les groupes qui peuvent se connecter via SSH. Exemple: `AllowUsers alice bob`
- **X11Forwarding**— Activer ou désactiver la redirection X11 (exécutant des applications d'interface graphique à distance). Exemple: `X11Forwarding yes`
- **LogLevel**— Contrôle la verbosité des journaux SSH. Exemple: `LogLevel VERBOSE`
- **ClientAliveInterval** et **ClientAliveCountMax**— Keepalive côté serveur pour détecter les connexions mortes.

- **Banner**— Fichier pour afficher un message avant la connexion.

**Note:**

Après avoir modifié la configuration du serveur, vérifiez la configuration `sudo sshd -t`, si ok alors redémarrez SSH `sudo systemctl restart sshd`

## Port Forwarding (Tunneling SSH)

Le transfert de **port SSH** (ou tunneling) nous permet d'envoyer en toute sécurité le trafic réseau via une connexion SSH cryptée. C'est comme créer un tunnel privé sécurisé entre notre machine locale et un serveur distant.

### Transfert de port local

Accédez à un service sur un réseau distant comme si c'était sur notre machine locale.

```
ssh -L 8080 :localhost : 80 nom d'utilisateur @remote_host
```

Cela fait avancer le **port 80** sur le `remote_host` au **port 8080** sur notre `localhost`. Visite <http://localhost:8080> dans notre navigateur nous montrera le serveur Web fonctionnant sur la machine distante.

### Port distant en avant g

Exposez un service sur notre machine locale à un réseau distant.

```
ssh -R 8080:localhost:3000 username@remote_host
```

Cela expose le service en cours d'exécution sur le **port 3000** de notre machine locale à **port 8080** sur le `remote_host`.

## Meilleures pratiques de sécurité

- **Désactiver l'authentification par mot de passe** — Une fois que nous avons des clés SSH configurées, nous devrions désactiver l'authentification par mot de passe entièrement dans le fichier de configuration SSH du serveur (`/etc/ssh/sshd_config`) par la mise en place `PasswordAuthentication no`.
- **Utilisez de fortes phrases de passe** – Protégez toujours nos clés privées avec une forte phrase de passe.
- **Modifier le port par défaut** — Le **port SSH** par défaut est **22**. Le changer en un port non standard peut réduire les attaques automatisées.
- **Désactiver la connexion root** — Empêcher la connexion directe comme le `root` utilisateur. Réglage `PermitRootLogin no` dans le `sshd_config` fichier.

- **Définir Les Autorisations Appropriées**

```
chmod 700 ~/.ssh # 700 (drwx-----)
chmod 600 ~/.ssh/id_ed25519 # 600 (-rw-----)
chmod 644 ~/.ssh/id_ed25519.pub # 644 (-rw-r---)
chmod 600 ~/.ssh/authorized_keys # 600 (-rw-----)
chmod 600 ~/.ssh/config # 600 (-rw-----)
```

## Dernières Pensées

**SSH** est un outil essentiel pour les développeurs et les administrateurs système non seulement pour se connecter à des serveurs, mais aussi pour pousser du code en toute sécurité vers les dépôts Git, gérer l'infrastructure à distance et transférer des fichiers.

**Dans cet article, nous avons:**

- Il a exploré ce qu'est **SSH** et pourquoi il est essentiel pour le développement moderne.
- A appris à se connecter à des serveurs distants, à transférer des fichiers et à utiliser l'authentification basée sur des clés en toute sécurité.
- Couvert comment gérer plusieurs clés et **configurations SSH** en utilisant le `~/.ssh/config` fichier.
- J'ai examiné l'utilisation avancée comme **l'agent SSH**, le transfert de port et la configuration côté serveur.

Nous pouvons durcir notre configuration SSH en mettant en œuvre des techniques avancées qui améliorent la sécurité, améliorent le contrôle d'accès et prennent en charge une meilleure évolutivité:

- **Fail2ban** — Bloque automatiquement les IP qui tentent de forcer notre connexion SSH.
- **Authentification à deux facteurs (2FA)** — Ajoutez une couche de sécurité supplémentaire sur l'authentification basée sur les clés.
- **Certificats SSH** — Pour gérer un grand nombre d'utilisateurs et de serveurs de manière plus sécurisée.
- **Pare-feu (par exemple, UFW, iptables, pare-feu)** — Limitez les adresses IP qui peuvent atteindre notre port SSH, réduisant ainsi l'exposition à un accès non autorisé.